

Samuel Antonio Castro Jaimes 202422244
Juan Pablo Diaz Salcedo 202420409

Informe Proyecto 3 – VeriLang

Fuente de la solución

Partimos de nuestra propia solución del proyecto 2. Los puntos 1 al 4 los completó Juan Pablo Díaz, y los puntos 5, 6 los realizó Samuel Castro.

Correcciones de problemas anteriores

Attribute en Syntax y AST no coincidían: En la sintaxis el valor del atributo era opcional, pero en el AST siempre se esperaba un valor. Se agregó un segundo constructor sin valor y se conectó el emptyAttributeValue() que ya existía pero no se usaba.

Arrow y LAngle como lexicals separados: Teníamos $\geq$ y $\leq$ definidos como tokens separados, lo cual complicaba el árbol de parse sin necesidad. Se reemplazaron por los literales directamente en las reglas.

SpaceParent guardaba el $<$ en el AST: El constructor **parent** tenía un campo **str angle** que siempre iba a ser " $<$ ", así que no servía para nada. Se eliminó ese campo.

Uso del Main:

Para correr este programa, abrir la terminal Rascal y ejecutar:

```
import Main;
```

```
main(|file:///RUTA/AL/ARCHIVO/ejemplo.vl|);
```

Ejemplo:

```
main(|file:///C:/Users/TuUsuario/ruta/al/proyecto/src/main/rascal/ejemplo.vl|);
```

Punto 1 – Parser

Se definió el parser del lenguaje usando la gramática de Syntax.rsc. El parser se genera automáticamente en Rascal a partir de las reglas sintácticas mediante la función `parse(#start[Program], file)`, que recibe la ubicación de un archivo .vl y produce un árbol de parse concreto. Este árbol es luego convertido al AST

tipado mediante `implode(#Program, pt)`. Para ejecutar el parser desde la consola de Rascal se usa el siguiente comando:

```
import Main;
main(|project://dsl-project/src/main/rascal/ejemplo.vl|);
```

Punto 2 – Generación de código

Se implementó el `Main.rsc` para poder correr programas VeriLang desde la consola. El flujo de ejecución es el siguiente:

1. El programa recibe la ubicación de un archivo `.vl` como argumento
2. Se parsea el archivo produciendo un árbol de parse concreto
3. Se convierte el árbol al AST tipado mediante `implode`
4. Se muestra el AST en consola
5. Se corre el verificador de tipos y se reportan los resultados

El AST mostrado en consola refleja fielmente la estructura del programa, cada definición del módulo aparece como un nodo del árbol con su tipo correspondiente.

Al ejecutar un programa válido, el output en consola es el siguiente:

```
[OK] Parse exitoso
[OK] AST construido

=== Resultado ===
program(moduleDef("MiModulo", [], [
  spaceDefItem(spaceDef("Enteros", noParent())),
  spaceDefItem(spaceDef("Naturales", parent("Enteros"))),
  operatorDefItem(operatorDef("suma", typeSig(typeAtom(intType()),
    [typeSigTail(typeAtom(boolType()))]), [])),
  variableDefItem(variableDef([
    varBinding("x", intType()),
    varBinding("y", boolType())
  ])),
  ruleDefItem(ruleDef(
    operatorApp("suma", [termId("x")]),
    operatorApp("suma", [termId("y")])
  )),
  structDefItem(structDef("MiLista", userType("Enteros"), [
    intVal("1"), boolVal("true"), charVal("'a'"), strVal("\"hola\"")
  ]))
]))
```

Punto 3 – Instalación de TypePal

Se integró TypePal al proyecto en dos pasos. Primero se agregó la dependencia en el pom.xml:

```
<dependency>
  <groupId>org.rascalmpl</groupId>
  <artifactId>typepal</artifactId>
  <version>0.14.8</version>
</dependency>
```

Después de agregar la dependencia se ejecutó mvn compile para descargar TypePal al proyecto.

Segundo, se creó el archivo Checker.rsc que extiende TypePal y lo conecta con la gramática de VeriLang. La estructura del checker sigue el patrón estándar de TypePal:

Tipos (AType): Se definieron los tipos del lenguaje que TypePal usará internamente para verificar el programa:

- intType() — Integer
- boolType() — Boolean
- charType() — Char
- strType() — String
- userType(str name) — tipos definidos por el usuario como spaces
- operatorType(AType from, AType to) — tipo de un operador curried

Roles (IdRole): Se definieron los roles que pueden tener los identificadores del programa:

- spaceId() — para espacios definidos con defspace
- operatorId() — para operadores definidos con defoperator
- variableId() — para variables definidas con defvar
- structId() — para estructuras definidas con defstruct

Funciones collect: Se definió una función collect por cada construcción del lenguaje. Estas funciones recorren el árbol de parse concreto y registran en TypePal las definiciones (c.define) y los usos (c.use) de cada identificador. Por ejemplo, cuando se encuentra un defspace Naturales < Enteros end, se registra Naturales como una definición de tipo spaceId() y Enteros como un uso que debe estar definido. Cuando se encuentra un defoperator suma : Integer -> Boolean end, se registra suma como un operador con tipo operatorType(intType(), boolType()).

El checker se invoca desde Main.rsc mediante:

```
TModel tm = veriLangTModel(pt);
```

Donde `pt` es el árbol de parse del programa. `TypePal` resuelve todas las definiciones y usos, y reporta los errores encontrados en `tm.messages`.

Punto 4 – Anotaciones de tipos en la gramática

Se actualizó la gramática en `Syntax.rsc` y el AST en `AST.rsc` para soportar tipos explícitos en el lenguaje. Los cambios fueron los siguientes:

Tipos primitivos: Se agregó una regla `Type` que reconoce los cuatro tipos primitivos del lenguaje más tipos definidos por el usuario:

```
rascal
```

```
syntax Type
```

```
= intType: 'Integer'
```

```
| boolType: 'Boolean'
```

```
| charType: 'Char'
```

```
| strType: 'String'
```

```
| userType: ID name;
```

Integración con operadores y variables: Se actualizó `TypeAtom` y `VarBinding` para usar `Type` en lugar de `ID`, permitiendo que operadores y variables usen tipos primitivos:

```
rascal
```

```
syntax TypeAtom = typeAtom: Type typeVal;
```

```
syntax VarBinding = varBinding: ID name ':' Type type;
```

Así un programa como el siguiente es válido:

```
defoperator suma : Integer -> Boolean end
```

```
defvar x : Integer y : Boolean end
```

Literales tipados: Se agregaron `lexicals` para los cuatro tipos de valores literales y una regla `TypedValue` que asocia cada valor con su tipo:

```
rascal
```

```
lexical IntegerLit = [0-9]+ !>> [0-9];
```

```
lexical BooleanLit = "true" | "false";

lexical CharLit  = "\"" [a-zA-Z0-9] "\"";

lexical StringLit = "\"" ![""]* "\"";
```

syntax TypedValue

```
= intVal: IntegerLit val ':' 'Integer'

| boolVal: BooleanLit val ':' 'Boolean'

| charVal: CharLit val ':' 'Char'

| strVal: StringLit val ':' 'String';
```

Estructuras de datos: Se agregó la construcción defstruct que permite declarar una estructura de datos con un nombre, un tipo dado por el usuario y una lista de valores tipados:

rascal

```
syntax StructDef = structDef: 'defstruct' ID name ':' Type structType TypedValue* values 'end';
```

Por ejemplo:

```
defstruct MiLista : Enteros
```

```
  1 : Integer
```

```
  true : Boolean
```

```
  'a' : Char
```

```
  "hola" : String
```

```
end
```

Esto se representa en el AST como:

```
structDef("MiLista", userType("Enteros"), [intVal("1"), boolVal("true"), charVal("'a'"),
                                             strVal("\"hola\"")]).
```

Punto 5 – Verificación de tipos

Se agregó en el **Checker.rsc** la verificación de que los tipos usados en el programa existan. Antes se podía escribir **defvar x : TipoInexistente end** y no pasaba nada. Ahora TypePal busca si ese nombre está definido como un **defspace** y si no lo encuentra, reporta un error.

Esto aplica en variables, operadores, structs y espacios padre.

Punto 6 – Elementos del struct

Se agregó una regla que verifica que los elementos dentro de un **defstruct** sean consistentes con el tipo declarado. Si el struct es de tipo primitivo (Integer, Boolean, Char o String), todos sus elementos deben ser de ese mismo tipo. Si el tipo es un space definido por el usuario, se permiten elementos mixtos.

Cambios en Main.rsc

Se agregó **import Checker** y la llamada a **veriLangTModel(pt)** para que al correr el programa también se ejecute la verificación de tipos y se impriman los errores encontrados.

Pruebas

Con un programa que usa **List** como tipo de struct sin haberlo definido:

[ERRORES] Problemas encontrados:
- error("Undefined space `List`", ...)

Con un programa correcto donde todos los tipos están definidos:

[ERRORES] Problemas encontrados:
ok